Database Technologies Lab Sheet — 7
Course Code: MCA105P

Lab Sheet • 10 Cursor Questions + 10 Trigger Questions • Questions Only

Part A – Syntax First

1. Cursor Syntax

```
DELIMITER //

CREATE PROCEDURE procedure_name()
BEGIN
   DECLARE done INT DEFAULT FALSE;
   DECLARE v_value datatype;

   DECLARE cur CURSOR FOR
      SELECT column_name FROM table_name;

   DECLARE CONTINUE HANDLER FOR NOT FOUND
      SET done = TRUE;

   OPEN cur;

   read_loop: LOOP
      FETCH cur INTO v_value;

      IF done THEN
         LEAVE read_loop;
      END IF;

      -- process v_value
   END LOOP;

   CLOSE cur;
END //

DELIMITER ;

CALL procedure_name();
```

A cursor processes a result set one row at a time. The NOT FOUND handler is used to detect the end of the cursor.

2. Trigger Syntax

```
DELIMITER //

CREATE TRIGGER trigger_name
```

```
AFTER INSERT ON table_name
FOR EACH ROW
BEGIN
   -- action
END //

DELIMITER ;
```

A trigger automatically executes when its defined table event occurs. Common events are INSERT, UPDATE, and DELETE.

3. NEW and OLD

```
-- INSERT: use NEW
-- UPDATE: use OLD and NEW
-- DELETE: use OLD
```

Use NEW for the incoming row and OLD for the previous/deleted row. In suitable BEFORE triggers, NEW values can be changed.

Part B – How to Run in MySQL: Step-by-Step

1. Open MySQL Command Line Client, MySQL Shell, or a MySQL-compatible SQL editor.

2. Log in to MySQL.

3. Create and select the lab database shown below.

4. Create both tables and insert the complete dataset before attempting the questions.

5. Verify the tables with SELECT * FROM Employees; and SELECT * FROM Employee_Audit;.

6. For procedures/cursors, type DELIMITER // before CREATE PROCEDURE and restore DELIMITER ; afterward.

7. For triggers, type DELIMITER // before CREATE TRIGGER and restore DELIMITER ; afterward.

8. Run cursor procedures with CALL procedure_name(...).

9. Test each trigger by performing the INSERT, UPDATE, or DELETE operation specified in the question.

10. Verify trigger definitions/results with SHOW TRIGGERS; and SELECT statements.

Part C – Lab Database and Dataset

Database: Hospital_Cursor_Trigger_Lab

```
CREATE DATABASE Hospital_Cursor_Trigger_Lab;
USE Hospital_Cursor_Trigger_Lab;

CREATE TABLE Doctors(
   DoctorID INT PRIMARY KEY,
   DoctorName VARCHAR(50),
```

```
    Department VARCHAR(40),
    ConsultationFee DECIMAL(10,2),
    Status VARCHAR(20)
);

INSERT INTO Doctors VALUES
(201,'Dr. Anil','Cardiology',1200,'Active'),
(202,'Dr. Beena','Pediatrics',900,'Active'),
(203,'Dr. Chirag','Neurology',1500,'Active'),
(204,'Dr. Divya','Dermatology',1000,'Active'),
(205,'Dr. Eshan','Cardiology',1300,'Active'),
(206,'Dr. Farah','Pediatrics',850,'Active');

CREATE TABLE Doctor_Audit(
    AuditID INT AUTO_INCREMENT PRIMARY KEY,
    DoctorID INT,
    ActionType VARCHAR(30),
    OldFee DECIMAL(10,2),
    NewFee DECIMAL(10,2),
    ActionTime TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Fee_Log(
    LogID INT AUTO_INCREMENT PRIMARY KEY,
    DoctorID INT,
    DoctorName VARCHAR(50),
    OldFee DECIMAL(10,2),
    NewFee DECIMAL(10,2),
    ChangedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Part D – Cursors: 10 Lab Questions

Question 1

Create a stored procedure using a cursor to display DoctorID and DoctorName for every doctor. Execute it and verify all six doctors are processed.

Question 2

Create a cursor procedure that displays doctors whose ConsultationFee is greater than 1000. Test the procedure.

Question 3

Create a cursor procedure with an OUT parameter to calculate the total ConsultationFee of all doctors. Call it and display the OUT value.

Question 4

Create a cursor procedure with an OUT parameter to count doctors belonging to the Cardiology department. Execute it and display the result.

Question 5
Create a cursor procedure to calculate the average ConsultationFee of all doctors. Handle the case where no rows are available.

Question 6
Create a cursor procedure that increases the ConsultationFee of all Pediatrics doctors by 10%. Execute it and verify the Doctors table.

Question 7
Create a cursor procedure that changes Status to 'Review' for doctors whose ConsultationFee is below 1000. Verify the affected rows.

Question 8
Create a cursor-based doctor report that processes rows ordered by Department and DoctorName and displays DoctorName, Department, and ConsultationFee.

Question 9
Create a cursor procedure with an OUT parameter to count all doctors whose Status is 'Active'. Execute it and display the count.

Question 10
Create a cursor procedure that gives a 5% fee increase to every doctor whose ConsultationFee is at least 1200. Execute it and verify the updated fees.

Part E – Triggers: 10 Lab Questions

Question 1
Create an AFTER INSERT trigger on Doctors that records the new DoctorID, action type, and new ConsultationFee in Doctor_Audit. Insert a test doctor and verify the audit table.

Question 2
Create an AFTER UPDATE trigger that records OLD and NEW ConsultationFee values whenever a doctor's fee changes. Update one doctor and verify Doctor_Audit.

Question 3
Create a BEFORE INSERT trigger that converts DoctorName to uppercase before the doctor is stored. Insert a test doctor using a lowercase name and verify the result.

Question 4
Create a BEFORE INSERT trigger that prevents a doctor from being inserted with a negative ConsultationFee. Use SIGNAL SQLSTATE '45000' with a meaningful message and test it.

Question 5
Create an AFTER DELETE trigger that records the deleted DoctorID and OLD ConsultationFee in Doctor_Audit. Delete a test doctor and verify the audit record.

Question 6
Create an AFTER UPDATE trigger that stores every changed consultation fee in Fee_Log, including DoctorID, DoctorName, OLD fee, and NEW fee. Test it with an UPDATE.

Question 7
Create a BEFORE UPDATE trigger that prevents ConsultationFee from becoming negative. Test the trigger with an invalid UPDATE.

Question 8
Create a BEFORE UPDATE trigger that normalizes a Status value of 'active' to 'Active'. Test it by updating a doctor.

Question 9
Create an AFTER UPDATE trigger that records an audit event when a doctor's Department changes. Change one department and verify the audit table.

Question 10
Create and test a trigger of your choice that demonstrates a useful hospital data-validation or auditing rule. Clearly